

PROGRAMAÇÃO DE BANCO DE DADOS

Prof. Rafael Tápio

ESTRUTURA E ELEMENTOS ENVOLVIDOS NA CRIAÇÃO DE TABELAS

O que é um Banco de Dados Relacional?

Um banco de dados relacional é um sistema de gerenciamento de banco de dados (SGBD) que armazena dados em tabelas estruturadas, onde cada tabela pode ter colunas e linhas, formando registros.

Tabelas: As tabelas são a estrutura básica em um banco de dados relacional. Elas armazenam dados de forma organizada.

Colunas e Linhas: Cada tabela possui colunas (também chamadas de atributos) que representam diferentes tipos de dados e linhas (ou registros) que contêm os dados.

Tabela / Linhas / Colunas

ID	NOME	IDADE	CIDADE	PAÍS
100	Maria	45	New York	EUA
101	José	32	São Paulo	Brasil
102	Paulo	79	Maringá	Brasil
103	Beto	17	Lisboa	Portugal

Tabela / Linhas / Colunas

ID	NOME	IDADE	CIDADE	PAÍS
100	Maria	45	New York	EUA
101	José	32	São Paulo	Brasil
102	Paulo	79	Maringá	Brasil
103	Beto	17	Lisboa	Portugal

Tabela

Tabela / Linhas / Colunas

ID	NOME	IDADE	CIDADE	PAÍS
100	Maria	45	New York	EUA
101	José	32	São Paulo	Brasil
102	Paulo	79	Maringá	Brasil
103	Beto	17	Lisboa	Portugal

LINHA = REGISTRO

Tabela / Linhas / Colunas

ID	NOME	IDADE	CIDADE	PAÍS
100	Maria	45	New York	EUA
101	José	32	São Paulo	Brasil
102	Paulo	79	Maringá	Brasil
103	Beto	17	Lisboa	Portugal

COLUNA = CAMPO

A Criação de uma Tabela

Para criar uma tabela no MariaDB, usamos o comando **CREATE TABLE**. A sintaxe básica para criar uma tabela é:

```
CREATE TABLE nome_tabela (  
    nome_coluna1 tipo_dado ,  
    nome_coluna2 tipo_dado ,  
    ...  
);
```

nome_tabela: O nome da tabela que será criada. Não pode ser repetido no mesmo banco de dados.

nome_coluna: O nome das colunas da tabela.

tipo_dado: O tipo de dado que a coluna pode armazenar (exemplo: INT, VARCHAR).

Para os exercícios da aula de hoje,
criem um database chamado, aula02.

Conjunto de caracteres: utf8mb4
Collation: utf8mb4_unicode_ci

```
CREATE DATABASE aula02  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;
```

```
USE aula02;
```

```
CREATE TABLE clientes (  
    id_cliente INT,  
    nome VARCHAR(100),  
    email VARCHAR(100)  
);
```

```
INSERT INTO clientes (id_cliente, nome, email)
VALUES (1, 'João Silva', 'joao.silva@email.com');
```

```
INSERT INTO clientes (id_cliente, nome, email)
VALUES (1, 'Maria Oliveira', 'maria.oliveira@email.com');
```

```
INSERT INTO clientes (id_cliente, nome, email)
VALUES (NULL, 'Carlos Souza', 'carlos.souza@email.com');
```

```
INSERT INTO clientes (id_cliente, nome)
VALUES (2, 'Ana Costa');
```

Analizando a tabela após as inserções

- Permitiu inserir todos os dados.
- Permitiu inserir sem id
- Permitiu duplicar o id
- Permitiu inserir sem email

123 id_cliente ▼	A-Z nome ▼	A-Z email ▼	
1	João Silva	joao.silva@email.com	
1	Maria Oliveira	maria.oliveira@email.com	
[NULL]	Carlos Souza	carlos.souza@email.com	
2	Ana Costa	[NULL]	

A Criação de uma Tabela com restrições

Para evitar problemas e melhorarmos a qualidade dos dados armazenados, temos a opção de colocarmos restrições nas tabelas:

```
CREATE TABLE nome_tabela (  
    nome_coluna1 tipo_dado [restrição],  
    nome_coluna2 tipo_dado [restrição],  
    ...  
);
```

restrição: Uma restrição opcional que define regras para os dados que podem ser inseridos na coluna (como NOT NULL, DEFAULT).

Restrições de Integridade - NOT NULL

NOT NULL:

Impede que a coluna aceite valores nulos.

Sintaxe:

```
nome_coluna tipo_dado NOT NULL
```

Exemplo:

```
nome VARCHAR(100) NOT NULL
```

Restrições de Integridade - PRIMARY KEY

PRIMARY KEY:

Define a chave primária da tabela. A chave primária é usada para identificar de forma única cada registro na tabela.

Sintaxe:

```
nome_coluna tipo_dado PRIMARY KEY
```

Exemplo:

```
id_cliente INT PRIMARY KEY
```

RECRIANDO A TABELA

Agora, vamos apagar a tabela e recriá-la com melhorias, aplicando restrições de chave primária e não nulo.

Existem comandos para alterarmos a tabela, que vamos aprender em outro momento.

```
DROP TABLE clientes;
```

```
CREATE TABLE clientes (  
    id_cliente INT PRIMARY KEY ,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL  
);
```

Explicação

id_cliente INT PRIMARY KEY:

- A coluna id_cliente agora é a chave primária da tabela.
- A chave primária garante que os valores desta coluna sejam únicos e não nulos. Ou seja, cada cliente terá um ID único.

nome VARCHAR(100) NOT NULL:

- A coluna nome agora não pode ser nula, ou seja, todo cliente precisa ter um nome. Isso evita que registros sejam inseridos sem um nome de cliente.

email VARCHAR(100) NOT NULL:

- A coluna email agora não pode ser nula. Ou seja, todo cliente precisa ter um email registrado.

```
INSERT INTO clientes (id_cliente, nome, email)
VALUES (1, 'João Silva', 'joao.silva@email.com');
```

```
INSERT INTO clientes (id_cliente, nome, email)
VALUES (1, 'Maria Oliveira', 'maria.oliveira@email.com');
-- Vai falhar porque 'id_cliente' precisa ser único (é chave primária).
```

```
INSERT INTO clientes (id_cliente, nome, email)
VALUES (NULL, 'Carlos Souza', 'carlos.souza@email.com');
-- Vai falhar porque 'id_cliente' não pode ser nulo (é chave primária).
```

```
INSERT INTO clientes (id_cliente, email)
VALUES (2, 'ana.costa@email.com');
-- Vai falhar porque 'nome' não pode ser nulo.
```

OUTRAS RESTRIÇÕES

Restrições de Integridade - **DEFAULT**

DEFAULT:

Define um valor padrão para uma coluna se nenhum valor for fornecido.

Sintaxe

nome_coluna tipo_dado DEFAULT valor

Exemplo:

data_criacao DATE DEFAULT CURRENT_DATE

Restrições de Integridade - CHECK

CHECK:

Impõe uma condição para os valores da coluna.

Sintaxe:

nome_coluna tipo_dado CHECK (condição)

Exemplo:

CHECK (idade >= 18).

```
DROP TABLE clientes;
```

```
CREATE TABLE clientes (  
    id_cliente INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    status VARCHAR(20) DEFAULT 'ativo',  
    idade INT CHECK (idade >= 18)  
);
```

Explicação:

status VARCHAR(20) DEFAULT 'ativo':

- A coluna status tem o valor padrão definido como 'ativo'.
- Se você inserir um registro e não fornecer um valor para status, ele automaticamente será atribuído como 'ativo'.

idade INT CHECK (idade >= 18):

- A coluna idade possui uma restrição CHECK, que garante que o valor inserido para essa coluna seja maior ou igual a 18.
- Ou seja, não será permitido inserir uma idade menor que 18.

```
INSERT INTO clientes (id_cliente, nome, email, status, idade)
VALUES (1, 'João Silva', 'joao.silva@email.com', 'ativo', 25);
```

```
INSERT INTO clientes (id_cliente, nome, email, idade)
VALUES (2, 'Maria Oliveira', 'maria.oliveira@email.com', 30);
```

```
INSERT INTO clientes (id_cliente, nome, email, idade)
VALUES (3, 'Carlos Souza', 'carlos.souza@email.com', 15);
-- Vai falhar porque a idade precisa ser maior ou igual a 18.
```

```
INSERT INTO clientes (id_cliente, nome, email, status, idade)
VALUES (4, 'Pedro Santos', 'pedro.santos@email.com', 'inativo', 28);
```

```
INSERT INTO clientes (id_cliente, nome, email)
VALUES (5, 'Lucas Lima', 'lucas.lima@email.com');
-- Vai funcionar, porque colocamos o check mas não o NOT NULL
```

```
DROP TABLE clientes;
```

```
CREATE TABLE clientes (  
    id_cliente INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    status VARCHAR(20) DEFAULT 'ativo',  
    idade INT NOT NULL CHECK (idade >= 18)  
);
```

```
INSERT INTO clientes (id_cliente, nome, email)
VALUES (5, 'Lucas Lima', 'lucas.lima@email.com');
-- Vai falhar, porque agora a idade não pode ser null
```

CHAVE PRIMÁRIA COMPOSTA

O que é uma Chave Composta?

Uma chave composta é uma **chave primária** formada por **duas ou mais colunas**. Ela é útil quando, individualmente, cada coluna pode ter valores repetidos, mas **a combinação de valores nas colunas deve ser única**.



```
CREATE TABLE matriculas (  
    id_aluno INT,  
    id_curso INT,  
    data_matricula DATE,  
    PRIMARY KEY (id_aluno, id_curso)  
);
```

Explicação

id_aluno e **id_curso** são as colunas da chave primária composta.

Isso significa que a combinação dos valores nessas duas colunas deve ser única. Ou seja, **um aluno não pode estar matriculado mais de uma vez no mesmo curso**.

data_matricula é uma coluna adicional que armazena a data da matrícula, mas não faz parte da chave primária.

```
INSERT INTO matriculas  
(id_aluno, id_curso, data_matricula)  
VALUES (1, 101, '2025-02-01');
```

```
INSERT INTO matriculas  
(id_aluno, id_curso, data_matricula)  
VALUES (1, 102, '2025-02-05');
```

```
INSERT INTO matriculas  
(id_aluno, id_curso, data_matricula)  
VALUES (2, 101, '2025-02-02');
```

```
INSERT INTO matriculas  
(id_aluno, id_curso, data_matricula)  
VALUES (1, 101, '2025-02-10');
```

EXERCICIOS

EXERCÍCIO - CHAVE COMPOSTA

1) Criação de Tabela com Chave Composta:

- Crie uma tabela chamada **produtos_loja**, onde a chave primária será composta pelas colunas **id_produto** e **id_loja**.
- A tabela também deve ter uma coluna **quantidade_estoque**, que armazena a quantidade de produtos disponíveis em estoque.

2) Inserir Dados:

- Insira ao menos 3 registros na tabela **produtos_loja** com diferentes combinações de **id_produto** e **id_loja**.
- Tente inserir um produto na mesma loja mais de uma vez para testar a restrição de chave composta.

```
CREATE TABLE produtos_loja (  
    id_produto INT,  
    id_loja INT,  
    quantidade_estoque INT,  
    PRIMARY KEY (id_produto, id_loja)  
);
```

-- Inserindo registros

```
INSERT INTO produtos_loja (id_produto, id_loja, quantidade_estoque)  
VALUES (1, 101, 100); -- Produto 1 na Loja 101
```

```
INSERT INTO produtos_loja (id_produto, id_loja, quantidade_estoque)  
VALUES (1, 102, 50); -- Produto 1 na Loja 102
```

```
INSERT INTO produtos_loja (id_produto, id_loja, quantidade_estoque)  
VALUES (2, 101, 200); -- Produto 2 na Loja 101
```

EXERCÍCIO: Criar uma Tabela de Produtos

Crie uma tabela chamada produtos para armazenar informações sobre produtos em estoque. A tabela deve conter as seguintes colunas:

- **id_produto** (inteiro, chave primária, não nulo).
- **nome** (texto com até 100 caracteres, não nulo).
- **preco** (número decimal, não nulo, deve ser maior que 0).
- **categoria** (texto com até 50 caracteres, valor padrão como "geral").
- **data_cadastro** (data, não nulo, com valor padrão como a data de inserção). - *(pesquise sobre **CURRENT_DATE**)*

A tabela deve garantir que:

- O preço seja sempre maior que 0.
- O id_produto seja único e a chave primária

```
CREATE TABLE produtos (  
    id_produto INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    preco DECIMAL(10, 2) NOT NULL CHECK (preco > 0),  
    categoria VARCHAR(50) DEFAULT 'geral',  
    data_cadastro DATE NOT NULL DEFAULT CURRENT_DATE  
);
```

EXERCÍCIO: Criar uma Tabela de Empregados

Crie uma tabela chamada empregados para armazenar informações sobre os funcionários de uma empresa. A tabela deve conter as seguintes colunas:

- **id_empregado** (inteiro, chave primária, não nulo).
- **nome** (texto com até 150 caracteres, não nulo).
- **salario** (número decimal, não nulo, deve ser maior que 1000).
- **departamento** (texto com até 50 caracteres, não nulo).
- **data_nascimento** (data, não nulo, deve ser uma data válida).
- **data_admissao** (data, não nulo, com valor padrão como a data de inserção).
- **status_empregado** (texto com até 10 caracteres, valor padrão como "ativo"). - O status_empregado deve ser um dos seguintes valores: 'ativo' ou 'inativo' (use a restrição CHECK e pesquise sobre "IN") - (pesquise sobre "IN" para colocar mais de uma condição)

```
CREATE TABLE empregados (  
    id_empregado INT PRIMARY KEY,  
    nome VARCHAR(150) NOT NULL,  
    salario DECIMAL(10, 2) NOT NULL CHECK (salario > 1000),  
    departamento VARCHAR(50) NOT NULL,  
    data_nascimento DATE NOT NULL,  
    data_admissao DATE NOT NULL DEFAULT CURRENT_DATE,  
    status_empregado VARCHAR(10) DEFAULT 'ativo' CHECK  
(status_empregado IN ('ativo', 'inativo'))  
);
```